Standard-Cell Characterization Methods for sg13g2_stdcell_hv

fo4.py (reference) · CharLib 2.1.0 · lctime 0.0.26 — the measurement physics behind the Liberty file

Koen Van Caekenberghe, Ph.D.

ChipDesign B.V.

info@chipdesign.be

2026-08-17

1 Scope and headline result

Three tools touched the `sg13g2_stdcell_hv` Liberty file, in three different roles:

- **`work/fo4.py`** — the *reference anchor*: a 96-line ngspice deck that measures the FO4 delay and the input capacitance of the thick-oxide `inv_1` against the thin-oxide original. Its two ratios (2.66× delay, 2.20× capacitance) set the slew and load grids of the shipped library, and its rail-biased AC capacitance (5.87 fF) is the reference every other capacitance number in this report is judged against.
- **CharLib 2.1.0** (commit `6859faf`, driven through the `infinitymdm` PySpice 1.6 fork) — the *production characterizer* that generated the shipped NLDM tables, configured with the `charge_integration` input- capacitance procedure and, for the sequential cells, the project's own `seq_delay_procedure.py` registered through CharLib's procedure registry.
- **lctime 0.0.26** (LibreCell) — the *independent cross-check*: `work/lctime_compare.py` re-characterized eight combinational cells on identical grids and models and aligned 3 132 table points against the shipped library.

Every method claim below was verified in the installed source of the three tools (file and line references throughout; a provenance appendix closes the report). The comparison in one table, before the derivations:

quantity	fo4.py (reference)	CharLib 2.1.0 as configured	lctime 0.0.26
delay / slew load	four real inverter gates	lumped capacitor	lumped capacitor
stimulus ramp	shaped by a driver stage	PWL, stretched by 1/0.6 (Liberty-correct)	PWL equal to the raw slew number (**convention error**)
extraction	ngspice `.meas trig/targ`	ngspice `.meas trig/targ`	numpy interpolation of waveforms
input capacitance			

quantity	fo4.py (reference)	CharLib 2.1.0 as configured	lctime 0.0.26
	AC at 1 MHz, rail-biased, mean of both rails	charge integration, worst edge	constant-current slope 10 µA, 20–80 % window
`inv_1` C_{in} result	5.87 fF	6.44 fF (+9.7 %)	8.92 fF (+52 %)
internal power	not measured	**not implemented in CharLib 2.1.0**	rectangle-rule energy integration (load energy not subtracted)
leakage	not measured	DC operating point per input state, all 2^N states	**unimplemented stub**
setup / hold	—	pass/fail bisection, 1.5× C2Q degradation (local procedure)	Brent root-finding, 10 ps absolute pushout
clock→Q	—	local procedure (upstream is a stub)	implemented; constraint search at zero output load
simulator coupling	`ngspice -b` batch	libngspice in-process (shared)	ngspice subprocess, ASCII `wrdata`

Answer to the standing question — can lctime use CharLib's charge-integration method? Not as shipped: lctime 0.0.26 contains no code path that integrates a current anywhere (no `trapz`, no `cumsum`, no ngspice `integ` in any generated `.control` block). Its input capacitance is a constant-current secant slope, which is where its +52 % error comes from. However, lctime's simulator coupling *already* returns branch currents with the time vector — it integrates supply and gate current for its internal-power tables — so a charge-integration capacitance is a localized, three-part patch to one function. Section 8 gives the exact changes.

2 What the Liberty NLDM asks for

All three tools ultimately serve the same data model, so the definitions come first. The library header fixes eight trip points; the shipped file uses the industry-standard set

$$V_{th,\,in} = V_{th,\,out} = 0.5\,V_{DD}, \qquad V_{slew,\,lo} = 0.2\,V_{DD}, \qquad V_{slew,\,hi} = 0.8\,V_{DD}$$

and defines, per timing arc, the propagation delay and the output transition time as threshold-crossing intervals of the simulated waveforms:

$$t_{pd} = t(v_{out} = 0.5\,V_{DD}) - t(v_{in} = 0.5\,V_{DD})$$
$$t_{tran} = t(v_{out} = 0.8\,V_{DD}) - t(v_{out} = 0.2\,V_{DD})$$

(rising conventions shown; falling edges mirror the thresholds). The NLDM `cell_rise` / `rise_transition` groups tabulate these two quantities as functions of two independent variables,

$$t_{pd} = f(s_{in}, C_L), \qquad t_{tran} = g(s_{in}, C_L),$$

where s_{in} is the *input* transition time measured between the same 20 %/80 % slew thresholds, and C_L the lumped load. The shipped library uses 7×7 grids per arc; the grid values are the thin-oxide library's grids scaled by the fo4.py ratios so the tables span the same electrical territory. Everything a characterizer does for a combinational arc reduces to filling f and g point by point from transient simulations, and every methodological difference between the three tools is a difference in *stimulus construction*, *load modelling*, or *waveform extraction* for those same two equations.

A physical scale for the delays themselves comes from logical effort: with $\tau = R_{inv}C_{in}$ the technology time constant, a stage driving an electrical effort $h = C_{out}/C_{in}$ has normalized delay

$$d = g\,h + p,$$

and the fanout-of-4 inverter delay ($g = 1$, $h = 4$) is the canonical technology speed metric fo4.py measures directly: $t_{FO4} = \tau\,(4 + p)$. It is a *ratio-preserving* metric — sizing every cell by the same factors leaves g and p nearly unchanged — which is exactly why one FO4 number per library (142.4 ps HV vs 53.6 ps LV) legitimately rescales an entire grid set.

3 Input capacitance: three answers to one question

3.1 The physics being sampled

The input pin of a CMOS gate is not a capacitor; it is a nonlinear, *transcapacitive* charge reservoir. The gate charge is a function of the gate voltage **and** of every other terminal voltage, so the differential capacitance seen while the input traverses its switching trajectory is

$$C_{in}(v) = \frac{dQ_G}{dv_{in}} = C_{gs}(v) + C_{gb}(v) + C_{gd}(v)\left(1 - \frac{dv_{out}}{dv_{in}}\right).$$

The last term is the Miller effect: while the cell switches, the output moves *against* the input with the incremental gain $A_v = dv_{out}/dv_{in} < 0$, so C_{gd} contributes $C_{gd}(1 + |A_v|)$ — a sharp peak centred on the switching threshold V_M, where $|A_v|$ is largest. Away from V_M, with the output pinned at a rail, $A_v \approx 0$ and C_{in} relaxes to its small end-point values.

Every "input capacitance" is therefore a *functional* of $C_{in}(v)$ — a weighted average over some window of the input swing — and the three tools choose three different windows. That single observation explains all three numbers.

3.2 fo4.py: small-signal AC at the rails

fo4.py drives an isolated inverter input with a 1 V AC source at 1 MHz, once biased at $v = 0$ and once at $v = V_{DD}$ (fo4.py:38–55). For a linear one-port $Y(j\omega) = G + j\omega C$, the measured current at unit drive gives

$$C_{AC} = \frac{\mathrm{Im}\,\{I\}}{2\pi f\,|\hat{V}|} \qquad \text{(evaluated at } f = 1\text{ MHz).}$$

The frequency is chosen so that gate leakage is invisible ($G \ll \omega C$: at 1 MHz, $\omega C \approx 3.7 \times 10^{-8}$ S for 5.9 fF, orders above any gate conductance) yet low enough that non-quasi-static channel effects play no role. The deck comments state the bias rationale explicitly: at either rail the inverter sits in a zero-gain region, so $A_v \approx 0$ and the Miller term vanishes —

$$C_{AC} = \tfrac{1}{2}\big[\,C_{in}(0) + C_{in}(V_{DD})\,\big] \quad \text{(endpoint sample, no Miller).}$$

A mid-rail bias would instead sample the peak of $C_{in}(v)$ and Miller-multiply C_{gd} into the answer. Result: **5.87 fF** for `sg13g2_hv_inv_1`.

3.3 CharLib: charge integration

The configured procedure (`charlib/characterizer/procedures/pin_capacitance/charge_integration.py`) drives the pin with a PWL *voltage* ramp — VSS up to VDD, wait, back down — and lets ngspice integrate the stimulus-source branch current over each edge window:

```
.meas TRAN q_rise integ i(vstim) from=... to=...
.meas TRAN q_fall integ i(vstim) from=... to=...
```

then applies the defining relation of effective capacitance (charge_integration.py:100–135):

$$C_{int} = \frac{|\Delta Q|}{\Delta V} = \frac{1}{V_{DD}} \int_{edge} i_{in}(t)\,dt = \frac{1}{V_{DD}} \int_0^{V_{DD}} C_{in}(v)\,dv.$$

This is the *full-swing mean* of $C_{in}(v)$ along the real switching trajectory: the Miller charge $Q_M \approx C_{gd}\,(\Delta v_{in} + \Delta v_{out})$ is included in ΔQ, but it is spread over the full denominator V_{DD}. Non-driven pins are isolated with 10 GΩ ∥ 1 pF; the ramp time defaults to the fastest grid slew and the wait time to 1000× that, so each edge integrates to completion. The reported `capacitance` is the worst (larger) of the two edges. Result: **6.44 fF** (+9.7 % vs the AC reference — that surplus *is* the Miller and nonlinearity charge, not an error; it is the charge a driving cell genuinely must deliver across a full transition).

Worth recording: CharLib's *upstream default* is a different procedure, `ac_sweep`, which fits capacitance as the slope of the admittance *magnitude* versus frequency, $C = d|Y|/df$. Since $|Y| = 2\pi f C$ for a capacitor, the correct estimator is $C = \frac{1}{2\pi}\,d|Y|/df$; the missing $1/2\pi$ makes the default overestimate by ≈ 6.28×, and the observed 6.8–7.5× (inv_1: 43.8 fF) is exactly 2π

compounded with the Miller contribution of its mid-transition drive. This is why the project configuration selects `charge_integration`.

3.4 lctime: constant-current secant slope

lctime does something else entirely (`lctime/characterization/input_capacitance.py`, docstring: *"Measurement of the input capacitance by driving the input pin with a constant current"*). A fixed source $I = 10\ \mu$A (hard-coded, util.py:162) charges the pin; the pin *voltage* alone is recorded; and the capacitance is the secant slope between the two slew trip points (input_capacitance.py:273–297):

$$C_{slope} = \frac{I}{\Delta V/\Delta t} = \frac{I\,[t(0.8V_{DD}) - t(0.2V_{DD})]}{0.6\,V_{DD}}.$$

From first principles this is again a windowed mean of the same $C_{in}(v)$: with $C_{in}(v)\,dv/dt = I$,

$$\Delta t = \frac{1}{I}\int_{0.2V_{DD}}^{0.8V_{DD}} C_{in}(v)\,dv \quad \Rightarrow \quad C_{slope} = \frac{1}{0.6\,V_{DD}}\int_{0.2V_{DD}}^{0.8V_{DD}} C_{in}(v)\,dv.$$

The window $[0.2, 0.8]\,V_{DD}$ *contains* the Miller peak at V_M and *excludes* the low-capacitance tails near the rails; the Miller charge is divided by $0.6\,V_{DD}$ instead of V_{DD}. Both effects push the same direction, which orders the three estimators from first principles alone:

$$C_{AC}\ \text{(endpoints, no Miller)}\ <\ C_{int}\ \text{(full swing mean)}\ <\ C_{slope}\ \text{(peak window m}$$

— and the measurements land in exactly that order: 5.87 < 6.44 < 8.92 fF, the slope method **+52 %** above the reference. lctime averages the result over all 2^n static states of the other inputs and over both edge directions (reduction per calc-mode: mean for `typical`), and carries a genuine bug while doing so: the returned dictionary swaps the labels, assigning the falling measurement to `rise_capacitance` and vice versa (input_capacitance.py:318–321). The default `capacitance` attribute, being the mean of both, hides the swap.

4 Delay and output slew

4.1 Stimulus: the slew convention is where the tools diverge

Liberty defines s_{in} between the 20 %/80 % thresholds. A characterizer that drives the pin with a single linear ramp of total duration T (0→100 %) realizes a Liberty slew of

$$s = (0.8 - 0.2)\,T = 0.6\,T \qquad \Longleftrightarrow \qquad T = \frac{s}{0.6}.$$

CharLib gets this right. Its `slew_pwl` helper (utils.py:49–65, explicitly citing the Liberty User Guide) stretches the requested slew to the full ramp, $T = s/(V_{hi} - V_{lo}) = s/0.6$, before building the PWL source.

lctime does not. Its `StepWave` for combinational stimulus is built with `rise_threshold=0`, `fall_threshold=1` (timing_combinatorial.py:186–191), i.e. the table's slew number is used directly as the 0→100 % ramp time. The circuit therefore sees an edge whose *Liberty* slew is only $0.6\,s$ — every lctime table point at index s is actually a measurement at $0.6\,s$. Since $\partial t_{pd}/\partial s_{in} > 0$, lctime under-reads delays, negligibly at fast edges and severely at slow ones. The cross-check quantifies it: over the STA-relevant region the two tools agree to a median 2.9 % on delays and 0.0 % on output transitions (3 132 points, 8 cells), but at the 3.36 ns slew point of `inv_1` (0.396 pF) a direct ngspice measurement gives 2.3605 ns where CharLib's table says 2.3571 ns (**+0.15 %**) and lctime says 1.9158 ns (**–19 %**); re-interpolating the CharLib table at $0.6\,s$ reproduces lctime to 0.25 %, nailing the mechanism. Ironically lctime *reads* all eight trip points from the Liberty template header (util.py:377–398) and measures with them correctly — only the stimulus construction ignores them.

fo4.py sidesteps the question: its device under test is driven by a real inverter stage (`Xdrv`), so the edge shape is whatever the technology produces, and the measured 50 % crossings use ngspice `.meas` with `rise=2/fall=2` — the *second* edge, past the initial-condition transient, so the measurement is taken in periodic steady state.

4.2 Load modelling

Both characterizers drive a lumped capacitor: C_L to ground on the output (CharLib delay.py: 83; lctime deck `Cload` elements). fo4.py loads the DUT with **four actual inverter inputs** — nonlinear $C_{in}(v)$ loads that also kick Miller charge back into the driving node. A lumped capacitor equal to $4\,C_{in}$ is the NLDM abstraction of that load; the FO4 deck is the ground truth the abstraction is checked against. This is precisely why the library's load grid is anchored to a *measured* C_{in} ratio rather than a nominal one.

Side-input handling differs in a way that shows up in multi-input cells: CharLib simulates every non-masking static state of the other inputs and records the **worst case**, `criterion = max` (delay.py:16, 188–190); lctime enumerates the unateness-consistent states and reduces with the calc-mode function — `np.mean` under the default `typical` mode (timing_combinatorial.py:77–81). Two tables built from identical simulations can therefore legitimately differ on any cell where the arc delay depends on the side-input state (stack position effects).

4.3 Extraction

fo4.py and CharLib let ngspice do the measurement — `.meas tran ... trig v(in) val=... targ v(out) val=...` — which interpolates crossings inside the simulator on the adaptive time grid. lctime post-processes: it normalizes the stored waveforms to $[0, 1]$, mirrors falling edges, and linearly interpolates the bracketing samples of each threshold crossing in numpy (util.py:251–374). Both are first-order interpolations of the same transient; the difference is not accuracy but *plumbing* — and the plumbing matters, as §7 shows: PySpice's subprocess backend silently drops `.meas` results, which is why CharLib must run in shared-library mode, while lctime never uses `.meas` at all and is immune.

5 Energy, power, leakage

5.1 The energy bookkeeping a Liberty file expects

Per rising output transition the supply delivers charge $Q = \int i_{VDD}\,dt$, hence energy

$$E_{supply} = V_{DD} \int i_{VDD}(t)\,dt = V_{DD}\,\Delta Q.$$

Of this, $C_L V_{DD}^2$ is associated with the external load (half stored on C_L, half dissipated in the pull-up network), and the remainder is *internal*: parasitic-node charging plus the short-circuit (crowbar) charge that flows while both networks conduct near V_M. Liberty's `internal_power` tables are meant to carry only that remainder, because the STA tool separately computes the switching energy of the external net from its own capacitance:

$$E_{int} = V_{DD} \int i_{VDD}\,dt - C_L V_{DD}^2 \quad \text{(rising edge)}.$$

CharLib 2.1.0 does not measure internal power at all. The package contains no procedure writing `internal_power`, `rise_power` or `fall_power` — energy appears only as a unit definition — and `Characterizer.analyse_cell` schedules exactly: pin capacitance, then delay + leakage (combinational) or the constraint procedures (sequential). The shipped Liberty consequently has no internal-power groups; any power-aware flow consuming it sees switching and leakage power only.

lctime is the only tool of the three that writes power tables, inside the same transient as the delay measurement (timing_combinatorial.py:263–274):

```
gate_energy   = np.mean(gate_current  * input_voltage) * dt
supply_energy = np.mean(supply_current * vdd) * dt
switching_energy = gate_energy + supply_energy
```

Two defects are visible from first principles. First, the quadrature: $\mathrm{mean}(f) \cdot (t_N - t_0)$ equals $\int f\,dt$ **only on a uniform time grid**; ngspice's adaptive stepping clusters points around the edges, where $i\,v$ is largest, so the sample mean over-weights the transition region. The unbiased estimator on the same data is the trapezoidal sum $\int f\,dt \approx \sum_k \frac{1}{2}(f_k + f_{k+1})(t_{k+1} - t_k)$, i.e. `np.trapz(f, time)` — available but unused. Second, the accounting: no $C_L V_{DD}^2$ subtraction is performed, so the tabulated "internal" energy includes the full load-charging energy and grows linearly with C_L; an STA tool then counts that energy a second time as switching power. Sequential cells get no power tables even from lctime (the supply current is retrieved in the FF harness but never integrated).

5.2 Leakage

Static power is a per-state quantity because subthreshold leakage depends exponentially on the bias of every device in a stack,

$$I_{leak} \propto e^{(V_{GS} - V_{th})/nV_T}(1 - e^{-V_{DS}/V_T}),$$

so a NAND2 leaks differently in each of its four input states (stack effect). CharLib does the canonical thing: one DC operating point per input combination, all 2^N of them, with

$$P_{leak}(\text{state}) = V_{DD}\,|I_{VDD}(\text{state})|$$

read from the supply branch (leakage_power.py:67, 86–91), emitting one conditioned `leakage_power { when : ... }` group per state. lctime's `characterize_leakage_power` is an empty stub, never called — no leakage data at all. For the sequential cells (which CharLib's combinational-branch leakage never reaches, and which have no unique DC operating point anyway), the project's `seq_leakage.py` measures a *transient average* instead: a reset pulse forces a known state, then $P = V_{DD}\,|\overline{i_{VDD}}|$ with the mean taken by `.meas tran AVG` over the settled window 40–60 ns — a documented single-state approximation.

6 Sequential constraints

6.1 The physics: pushout

As the data edge approaches the clock edge, a flip-flop's internal regeneration starts from an ever-smaller initial imbalance and the clock-to-Q delay diverges logarithmically (regeneration time constant τ_m of the cross-coupled pair):

$$t_{C2Q}(t_{su}) \approx t_{C2Q,\infty} + \tau_m \ln \frac{t_0}{t_{su} - t_{su}^*}.$$

A setup (hold) constraint is therefore not a hard edge but a chosen point on this curve, and every characterizer must pick a *criterion*: how much delay pushout $\Delta d = t_{C2Q} - t_{C2Q,\infty}$ is tolerable. Inverting the curve shows how the criterion maps to the constraint:

$$t_{su}(\Delta d) = t_{su}^* + t_0\, e^{-\Delta d/\tau_m},$$

so a tighter (smaller) Δd yields a larger, more conservative setup time — but only logarithmically slowly, which is why differing criteria still produce comparable libraries.

6.2 CharLib: what is stock and what is local

Stock CharLib 2.1.0 is largely unimplemented here: the sequential *delay* procedure returns immediately (`delay.py:14–15`, a `# TODO` stub), as do recovery, removal, min-pulse-width and the metastability binary search. Its one working constraint procedure is the C2Q *contour* method (per the setup/hold-interdependence literature): reference C2Q at a relaxed corner, validity criterion $t_{C2Q} \leq 1.2 \times t_{C2Q,ref}$, exponential bracketing plus binary search for the window edges, then an N×N sweep of the (setup, hold) rectangle and knee-point selection on the pass contour. On these HV cells the contour harness built transients that exhausted memory, so the shipped library uses the project's procedures (`seq_delay_procedure.py`, registered through CharLib's own registry):

- **clock→Q**: preload pulse, data switch mid-period, measured clock edge at the grid slew; ngspice `.meas` from the clock's 50 % crossing (second rise) to the output's 50 % crossing, 20–80 % output transition; tables indexed clock-slew × load.
- **setup/hold**: pass/fail **bisection** on a two-pulse harness, search range −2…+8 ns, tolerance 10 ps. The pass criterion is deliberately two-part: the output must reach the target final state **and** its first half-rail crossing must occur within $1.5\times$ the nominal C2Q ($\Delta d = 0.5\, t_{C2Q,\infty}$, a *relative* degradation criterion) — final-state-only acceptance would silently ride through metastability. Latches are constrained against the closing enable edge; non-convergence counts as fail.

6.3 lctime

lctime's sequential path is genuinely implemented and more mathematically polished: it measures the pushout curve itself. `find_min_data_delay` doubles the setup/hold window until t_{C2Q} converges to $t_{C2Q,\infty}$ (abstol 1 ps); the constraint is then the root of

$$t_{C2Q}(t_{su}) - (t_{C2Q,\infty} + \Delta d) = 0, \qquad \Delta d = 10\ \text{ps (default max\_pushout\_time)},$$

found with Brent's method (`optimize.brentq`, xtol = 1 fs) after exponential bracketing — an *absolute* pushout criterion, in contrast to CharLib-local's relative one. It then re-solves setup with hold fixed at its unconditional minimum plus a 10 ps margin (and vice versa), which is what populates the constraint tables. Min-pulse-width uses the same root-finding on a two-pulse harness. Caveats visible in the source: the constraint search runs at **zero output load** (flagged `# TODO` in flipflop.py:196), clk→Q tables are taken at a fixed generous setup = hold = 1 ns rather than the measured constraints, and **latches are not supported at all** (main_lctime.py aborts) — where the shipped library needed en→Q and closing-edge constraints for its five latches.

For this library's C2Q of a few hundred ps, CharLib-local's $\Delta d = 0.5\, t_{C2Q,\infty}$ (~100+ ps) versus lctime's 10 ps sit far apart on the pushout curve, yet by the logarithmic inversion above the extracted constraints differ only by $\tau_m \ln(\Delta d_1/\Delta d_2)$ — a few regeneration time constants, i.e. tens of ps. The bigger structural difference is coverage: only the local CharLib procedures produced latch data at realistic loads.

7 Numerics and simulator coupling

All three tools run the same engine — ngspice with the IHP OSDI-compiled PSP103 Verilog-A MOS models — through three different couplings, and two of the project's hardest-won findings are couplings, not physics:

	fo4.py	CharLib	lctime
coupling	`ngspice -b` batch		

	fo4.py	CharLib	lctime
		libngspice in-process (PySpice fork)	subprocess, ASCII `wrdata`
`.meas` support	native	fork-added; **lost in subprocess mode**	not used
integration method	trapezoidal (default)	trapezoidal, `trtol = 1`	trapezoidal (no options set)
timestep	20 ps fixed print step	slew/8 (delay), slew/10 (cap)	10 ps default, `stop` when breakpoints
stop condition	fixed 3 periods	`autostop` on `.meas` completion	breakpoint at 1 %/99 % V_{DD}
OSDI model loading	`.spiceinit` (batch honors it)	`.spiceinit` copied into run dir	`.spiceinit` copied by compare script
temperature	deck value	25 °C	**hard-coded 25 °C** (liberty header ignored)

The trapezoidal rule's local truncation error per step, $\varepsilon \approx \frac{h^3}{12}\left|d^3v/dt^3\right|$, is what ngspice's timestep control bounds; `trtol` scales the tolerance it is compared against, so CharLib's `trtol = 1` (versus the default 7) forces $\approx 7^{1/3} \approx 1.9\times$ finer steps through the transitions — a deliberate accuracy/runtime trade in the delay decks.

The two coupling landmines, both documented in `run_charlib.sh` and the config generator because both produce *silently* wrong libraries: (1) with the `ngspice-subprocess` backend PySpice parses only the rawfile and never reads measurement results back, so every delay table comes back empty while leakage and capacitance still populate — the file looks plausible until a timing tool finds no arcs; shared mode is mandatory. (2) ngspice reads `.spiceinit` in batch and interactive mode but *not* in server mode (`ngspice -s`), so PySpice-driven runs never load the OSDI PSP103 models without the project's `pre_osdi` shim. lctime's file-based batch backend is immune to both by construction — its weakness is instead that it sets no simulator options at all and inherits whatever `.spiceinit` the working directory supplies, and that it stamps `.option TEMP=25` regardless of the library header (the `nom_temperature` it parses is never propagated; characterizing a non-25 °C corner silently produces 25 °C data).

8 Can lctime use the charge-integration method?

As shipped in 0.0.26 — no. The verdict rests on three code facts:

1. The input-capacitance procedure is irreducibly the constant-current slope method: it drives the pin from a current source, records *only* the pin voltage (`output_currents=[]`, input_capacitance.py:253–257), and computes $C = I\,\Delta t/\Delta V$ from two threshold crossings (lines 273–297). There is no charge in sight.

2. No code path in the package integrates a current: a search over the installed tree finds no `trapz`/`cumsum` and no ngspice `integ` in any generated control block.
3. The 10 µA source magnitude is a hard-coded constant (util.py:162), not reachable from the CLI — so even the existing method cannot be tuned, let alone replaced, by configuration.

But the port is small, because the data path already exists. For its power tables lctime already requests branch currents through named voltage sources and gets them back aligned with the time vector — both backends implement it (`wrdata ... i(vpin)` file-based; `print i(vpin)` interactive) — and the combinational routine already retrieves the *gate* current `-currents["V<pin>"]` for its energy term. Charge-integration capacitance is the same measurement pointed at a different source. The patch, confined to `characterize_input_capacitances()`:

1. **Drive voltage, not current**: declare the active pin as a voltage input and replace the `I<pin>` PWL current source with the `StepWave` ramp the delay code already uses (using the Liberty-correct $T = s/0.6$ stretch while at it).
2. **Ask for the current**: `output_currents=["V<pin>"]` instead of the empty list.
3. **Integrate**: replace the secant-slope block with

$$Q = \int_{edge} i_{pin}(t)\,dt \;\approx\; \texttt{np.trapz}(-i_{Vpin},\, t), \qquad C = \frac{|Q|}{V_{DD}},$$

using the trapezoidal sum (not the rectangle rule the power code uses) since the time grid is adaptive.

Nothing else in the harness changes — grids, state enumeration, reduction, and Liberty emission all operate on the returned scalar. On the evidence of this library, the payoff is the difference between +52 % and +9.7 % on `inv_1`'s pin capacitance, i.e. between a wire-load estimate that is wrong by half and one within the Miller-charge ambiguity that any single-number capacitance carries. (lctime is AGPL-3.0-or-later; a modified copy used in-house carries no distribution obligation, and an upstream contribution would resolve the license question entirely while fixing the rise/fall label swap noted in §3.)

9 Provenance of the shipped Liberty: no lctime data

A natural question given three tools: which of them actually populated `lib/sg13g2_stdcell_hv_typ_3p30V_25C.lib`? The answer is that every number in the shipped file traces to the CharLib flow, and **none to lctime**:

Liberty content	produced by
combinational `cell_rise/fall`, `rise/fall_transition` tables	CharLib 2.1.0 (`run_charlib.sh`), post-processed by `fix_lib.py`
pin `capacitance` / `rise_capacitance` / `fall_capacitance`	CharLib `charge_integration` procedure

Liberty content	produced by
per-state `leakage_power` groups (combinational)	CharLib all-states DC enumeration
clk→Q and en→Q arcs, setup/hold constraint tables	local `seq_delay_procedure.py` inside CharLib's registry, merged by `merge_lib.py`, fixed by `fix_lib_seq.py`
sequential `cell_leakage_power`	local `seq_leakage.py` (transient average)
slew and load index grids	thin-oxide grids × the fo4.py ratios (2.66 delay, 2.20 capacitance)
functions, pin directions, `ff`/`latch` groups	lifted from the thin-oxide Liberty (logic unchanged by the transform)

lctime's role was strictly read-only verification: `lctime_compare.py` re-characterizes eight combinational cells on the same grids and models and compares its output *against* the shipped tables — the 2.9 % median delay agreement, the −19 % slew-convention band and the +52 % pin-capacitance finding of the preceding sections are all products of that comparison, and none of its data flows back. Given what the comparison revealed about the slope-method capacitance (§3) and the slew convention (§4), that separation is the correct engineering outcome, not an accident of history. A text search of the shipped file confirms it carries no lctime or LibreCell traces.

10 Conclusions

1. **All three input-capacitance methods are averages of the same $C_{in}(v)$ over different windows**, and their results order exactly as the windows predict: rail end-points 5.87 fF < full-swing charge mean 6.44 fF < Miller-peak window mean 8.92 fF. Charge integration is the defensible production choice — it measures the charge a driver actually delivers — with the rail-biased AC value as the clean lower anchor.
2. **CharLib is Liberty-correct on the slew convention; lctime is not.** The $T = s/0.6$ ramp stretch is the single largest systematic between the two characterizers (−19 % at multi-ns slews, mechanism confirmed by re-interpolation at $0.6\,s$ to 0.25 %); where the convention doesn't bite, the tools agree to a median 2.9 % on delays and 0.0 % on transitions.
3. **Neither tool covers the full Liberty power model.** CharLib 2.1.0 has no internal-power procedure at all; lctime writes power tables but with a biased quadrature and without subtracting the $C_L V_{DD}^2$ load term, so they double-count against STA switching power. Leakage is CharLib-only (all-states DC enumeration; the project adds transient single-state leakage for sequential cells).
4. **Sequential characterization shipped only because of local code**: upstream CharLib's sequential delay is a stub and its contour method was unusable here; the local bisection (relative 1.5× C2Q criterion) and lctime's Brent-on-pushout (absolute 10 ps) are both principled points on the same metastability curve, differing mainly in coverage — lctime cannot do latches and constrains at zero load.

5. **lctime can adopt CharLib's charge-integration method with a three-part, single-function patch** — the branch-current infrastructure it needs is already exercised by its own power measurement. Until then, its pin capacitances should not be used for anything load-sensitive.

---

11 Appendix: provenance

item	identity
reference deck	`work/fo4.py`, 96 lines, ngspice batch
CharLib	2.1.0, git `stineje/CharLib` commit `6859faf`, venv `/foss/tools/charlib`, install hash-verified unmodified
PySpice	1.6 fork, git `infinitymdm/PySpice` commit `da81c4d` (adds `.meas` readback)
lctime	0.0.26, `/usr/local/lib/python3.12/dist-packages/lctime`
models	IHP SG13G2 PSP103 Verilog-A via OSDI, `cornerMOShv.lib mos_tt`, 3.3 V, 25 °C
shipped library	`lib/sg13g2_stdcell_hv_typ_3p30V_25C.lib`, thresholds 20/80/50, 7×7 NLDM grids
cross-check	`work/lctime_compare.py`: 8 cells, 3 132 aligned points
local CharLib adaptations	`charlib_patched.py` (case-insensitive branch lookup, procedure registration), `seq_delay_procedure.py` (clk→Q, setup/hold bisection), `seq_leakage.py`, `gen_charlib_config.py` (grids ×2.66/×2.20, charge integration selected), `fix_lib.py` / `fix_lib_seq.py` (header and sequential emission repairs)

Key source locations cited: CharLib `procedures/combinational/delay.py` (stimulus 75–83, measurements 101–112, worst-case 188–190), `pin_capacitance/charge_integration.py` (integration 100–104, formula 124–135), `combinational/leakage_power.py` (67, 86–91); lctime `characterization/timing_combinatorial.py` (StepWave 186–191, energy 263–274), `characterization/input_capacitance.py` (current source 235–243, slope 273–297, label swap 318–321), `characterization/timing_sequential.py` (pushout root-finding 1018–1228), `ngspice_subprocess.py` (deck 122–187, temperature 83).